PitchMind

Multi-Agent Football Insights Engine on Azure

Implementation Plan – Microsoft × Premier League *Inside the Game* Hackathon

Version 1.0 · 25 October 2026 · Working name, final name chosen last

Hard deadline: 27 October 2026, 11:59 pm Pacific Time = Tuesday 28 October, 07:59 Tunisia time (Official Rules, Section 1). Teams of up to 4. A project can win at most one Grand Prize and one Category Prize. Our internal target is to **submit by Monday 27 October, 23:00 Tunisia time**, keeping the night as buffer. All times in this document are Tunisia time. A feature that is not deployed and not visible in the 2-minute video scores zero.

Contents

1 Requirements Restatement **3**
- 1.1 What we are building . . . 3
- 1.2 Mandatory pipeline (category adherence) . . . 3
- 1.3 The 9 areas we cover . . . 3
- 1.4 Submission requirements . . . 3

2 What We Are Optimising For **4**
- 2.1 Judging rubric (5 × 20%) . . . 4
- 2.2 Side prizes we target . . . 4

3 Architecture **5**
- 3.1 System overview . . . 5
- 3.2 Key design decisions . . . 6
- 3.3 Latency budget . . . 6
- 3.4 Services . . . 6
- 3.5 Data stores . . . 7

4 Technology Stack **7**

5 Design Patterns **7**
- 5.1 Architecture patterns . . . 7
- 5.2 Code patterns (GoF) . . . 8
- 5.3 Agentic patterns . . . 8

6 Data Contracts (frozen first) **8**

7 Repository Structure **9**

8 Workstreams **9**

9 Implementation Phases **9**
- 9.1 Phase 0 – Foundations (Sat 25 Oct, 11:30–14:00) . . . 10
- 9.2 Phase 1 – Core engines in parallel (Sat 25 Oct, 14:00 – Sun 02:00) . . . 10
- 9.3 Phase 2 – Cloud and experience (Sun 26 Oct, 08:00–18:00) . . . 11
- 9.4 Phase 3 – Hardening (Sun 26 Oct, 18:00–23:00) . . . 12
- 9.5 Phase 4 – Submission (Mon 27 Oct) . . . 12

10 Demo Video Script (under 2:00) **12**

11 Testing and Validation **12**

12 Risks **13**

13 Complexity Estimate **13**

14 Definition of Done **14**

1 Requirements Restatement

1.1 What we are building

An AI system running on Azure that receives **synthetic** football events live, turns them into statistics, detects the moments that matter, explains *why* they matter with evidence, and delivers that intelligence as timed on-screen overlays and a second-screen app that is **personalized** per fan (persona, language, favourite club, favourite player).

1.2 Mandatory pipeline (category adherence)

#	Stage	How PitchMind does it
1	Ingest	Simulator (and CV module) publish `MatchEvents` to Azure Event Hubs in real time
2	Interpret	Stats Engine computes pass quality, speeds, xG, pressure, momentum, control-vs-chaos
3	Explain	Moment Detector + Explain Agent: why it matters, every claim cites event IDs and stats
4	Render	Render Service emits timed overlay JSON, pushed via Azure Web PubSub
5	Personalize	Persona + Language agents produce one variant per audience segment

1.3 The 9 areas we cover

Area	Scope	Priority
Synthetic data	Match simulator, fictional league, seeded demo match, published dataset	**MUST**
Real-time stats	Pass distance/accuracy/difficulty, ball & shot speed, player speed/distance, xG, pressure, momentum	**MUST**
Explainability	Turning points, pressure shifts, rhythm, control vs chaos, evidence citations	**MUST**
Narrative generation	Live commentary, storylines, milestones, half-time and full-time recaps	**MUST**
Multi-language	EN, FR, AR (right-to-left), ES; tone adapted, not just translated	**MUST**
Personalization	Analyst / Casual / Beginner, favourite club, player-focus, one-metric mode	**MUST**
Rendering & overlays	Timed overlay JSON, broadcast overlay page, second-screen app, studio dashboard	**MUST**
Cloud architecture	Event Hubs, Container Apps, Functions, Web PubSub, Cosmos DB, Foundry, Bicep, CI/CD	**MUST**
Computer vision	Short own-footage clip → YOLO tracking → ball speed event into same pipeline	**SHOULD**

1.4 Submission requirements

- Newly built project (we start on 25 October – compliant).
- Public GitHub repository.

- Demo video **under 2 minutes**, public on YouTube or Vimeo, showing the project running. No third-party trademarks, real club names/logos or copyrighted music.
- Short pitch: what we built, which Microsoft/Azure technologies, what problem it solves.

2 What We Are Optimising For

2.1 Judging rubric (5 × 20%)

Criterion	How we score it
Technological implementation	Clean monorepo, frozen contracts, typed code, tests, CI, Bicep, documented README
Agentic design & innovation	Specialised agents with distinct roles, shared state, handoffs, Verifier, failure recovery
Real-world impact	Actually deployed on Azure, latency budget met, fallbacks, observability, cost model
UX & presentation	Polished second-screen app, RTL Arabic, and a tight 2-minute video
Category adherence	The video walks the 5 stages in order and names each one

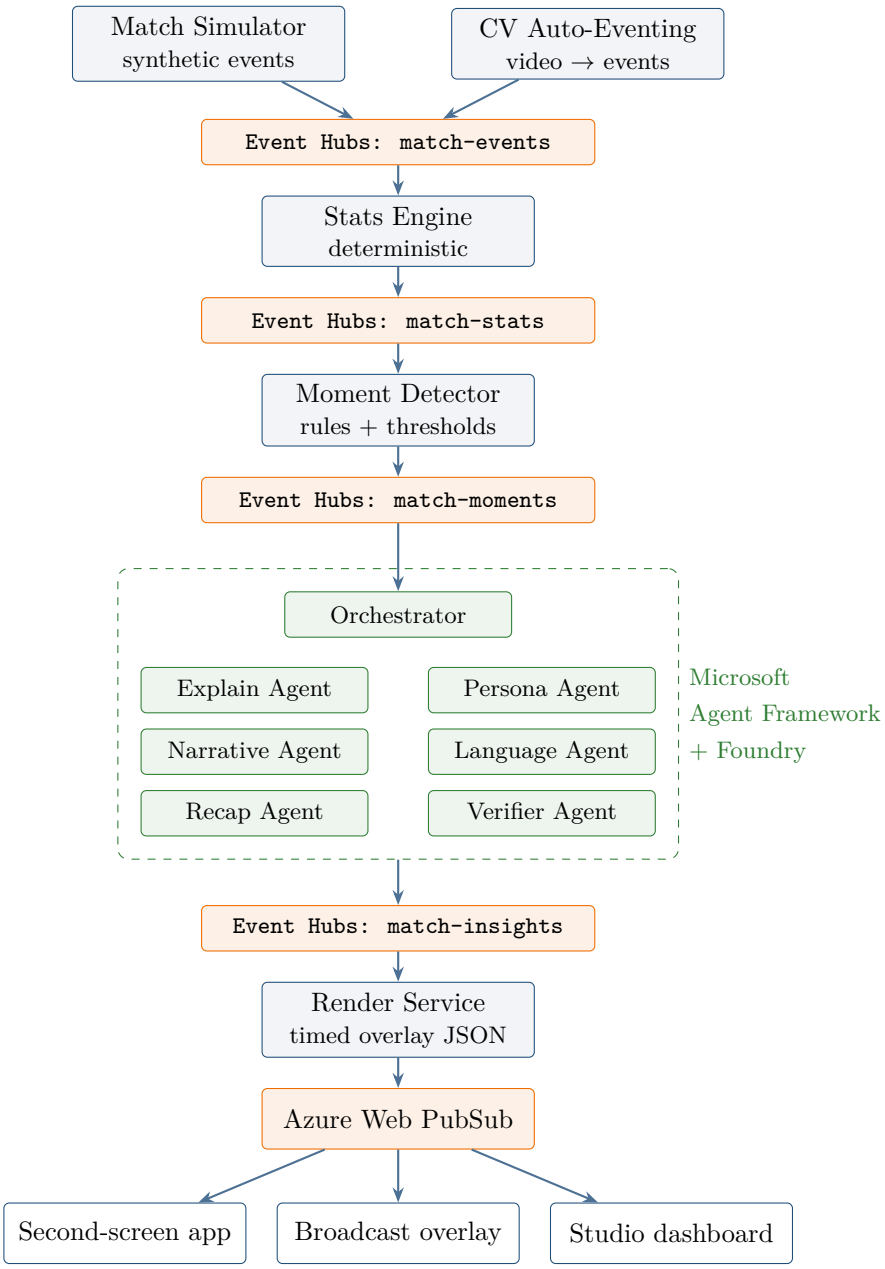
2.2 Side prizes we target

Prize	What we must show
Best Multi-Agent Orchestration	Microsoft Agent Framework workflow, shared match state, handoffs, live failure recovery demo, optional A2A endpoint
Best Use of Microsoft Foundry	Agents and models hosted in Foundry, Foundry tracing, Foundry evaluations, Content Safety
Best Azure Cloud Native	Event-driven: Event Hubs, Container Apps with KEDA scaling, Functions, Web PubSub, Bicep, CI/CD
Best Enterprise Solution	Evidence citations, Verifier, audit log, human-in-the-loop approval, Managed Identity, Key Vault, App Insights

3 Architecture

3.1 System overview

Event-driven microservices. Every stage reads one stream and writes the next. Statistics and moment detection are **deterministic code**; LLM agents are used only for language: explaining, narrating, personalising, translating and verifying.



3.2 Key design decisions

Decision	Reason
Stats and detection in plain code	Fast enough for live play, no hallucinated numbers, cheap
One `MatchEvent` schema	Simulator, CV or a future real feed plug in through an Adapter
Replayable event log	Demos, recaps, tests and debugging from the same stream
Personalize per *segment*	Segment = (persona, language, club, player). 10 000 fans ≈ 200 LLM calls, not 10 000
Every insight cites evidence	Event IDs and metric values attached; the Verifier checks them
Template fallback	If an LLM call fails or exceeds its budget, a template text is published; the overlay never goes blank
Transport abstraction	Local dev uses an in-memory/Redis bus; cloud uses Event Hubs. Same interface

3.3 Latency budget

Output	Target	Path
Stat overlays (player tag, speed, pass difficulty)	≤ 2 s	event → stats → render (no LLM)
Explanation / commentary	≤ 8 s	+ moment → agents → verifier
Personalized variants	≤ 10 s	parallel fan-out per active segment
Half-time / full-time recap	≤ 30 s	triggered by match phase change

3.4 Services

Service	Lang	Responsibility
`simulator`	Python	Generates seeded realistic matches; streams events at 1×–20× speed; exports JSONL dataset
`cv-eventing`	Python	YOLO + ByteTrack on a short clip, homography to metres, emits ball-speed / pass events
`stats-engine`	Python	Event-sourced match state; all metrics; publishes `StatUpdate`
`moment-detector`	Python	Chain of rules over stats; publishes `Moment` with evidence
`agents`	Python	Agent Framework workflow: orchestrator + 6 agents; publishes `Insight`
`render`	Python	Builds `Overlay` payloads, throttles, prioritises, schedules `show_at`
`api`	Python	FastAPI BFF: matches, profiles, replay, segment subscription, Web PubSub tokens, audit log
`web`	TS	Next.js second-screen app + studio dashboard
`overlay`	TS	Transparent broadcast graphics page (browser source)

3.5 Data stores

Store	Content
Azure Event Hubs	4 hubs: `match-events`, `match-stats`, `match-moments`, `match-insights`; retention for replay
Azure Cosmos DB	Serverless. Match state snapshots, insights, overlays, fan profiles, audit log
Azure Cache for Redis	Optional. Hot state and per-segment cache; in-process cache if time is short
Azure Blob Storage	Datasets, video clips, generated recaps
Azure AI Search	Player and club knowledge base for milestones (RAG)

4 Technology Stack

Layer	Language	Tools
Simulator, stats, detection	Python 3.12	NumPy, pandas, Pydantic v2, asyncio, `azure-eventhub`
xG model	Python	scikit-learn logistic regression trained on synthetic shots
Computer vision	Python	Ultralytics YOLO, OpenCV, `supervision` (ByteTrack)
Agents	Python	Microsoft Agent Framework, Foundry Agent Service, Azure OpenAI models in Foundry
API	Python	FastAPI, Uvicorn
Frontend	TypeScript	Next.js, React, Tailwind, Framer Motion, Recharts
Real-time push	—	Azure Web PubSub (client SDK in browser)
Infrastructure	Bicep	Azure Container Apps, Functions, ACR, Key Vault, Managed Identity
CI/CD	YAML	GitHub Actions: lint, test, build images, deploy
Quality	—	ruff, mypy, pytest; ESLint, Prettier, Playwright
Observability	KQL	Application Insights, Foundry tracing (OpenTelemetry)

Rule of thumb: Python behind the screen, TypeScript on the screen.

5 Design Patterns

5.1 Architecture patterns

Pattern	Where
Event Sourcing	Match state = fold over the event log; enables replay and recaps
CQRS	Writes go to streams; read models per client (fan app, studio, overlay)
Publish/Subscribe	All services communicate only through Event Hubs topics
Pipes and Filters	events → stats → moments → insights → overlays
Backend-for-Frontend	`api` shapes data per client
Cache-Aside	Personalized insights cached by segment key
Circuit Breaker + Fallback	LLM timeouts trip to template output

5.2 Code patterns (GoF)

Pattern	Where
Observer	Threshold watchers on stats (sprint > 33 km/h → player tag)
Strategy	Personas, tone styles, metric calculators, swappable at runtime
Adapter	Simulator / CV / future feed → `MatchEvent`; local bus / Event Hubs
Factory	Event creation in simulator; agent construction from config
State	Match phases: pre-match, 1st half, half-time, 2nd half, full-time
Chain of Responsibility	Moment-detection rules
Template Method	Recap skeleton (context, key moments, players, verdict) for HT and FT
Builder	Overlay payload construction
Decorator	Language, tone and club-focus layers over a base insight

5.3 Agentic patterns

Pattern	Where
Orchestrator–Worker	Orchestrator routes each `Moment` to the right agents
Shared state (blackboard)	Agents read the same match state through tools, not prompt dumps
Tool calling	`get_stats_window`, `get_events`, `get_player_profile`, `search_knowledge`
RAG	Milestones and player history from Azure AI Search
Grounding with citations	Every claim carries event IDs and metric values
Verifier	Rule check + LLM judge; numeric claims compared to stats; reject → one retry → template
Structured output	All agents return JSON validated against the contracts
Human-in-the-loop	Studio dashboard can hold/approve insights before broadcast

6 Data Contracts (frozen first)

All services depend on `contracts/`. Pydantic models are the source of truth; JSON Schemas and TypeScript types are generated from them. **Contracts are frozen at Milestone M1**; any later change needs the whole team's agreement.

Contract	Key fields
`MatchMeta`	match_id, teams, lineups, players (id, name, number, position, attributes), venue, seed
`MatchEvent`	id, match_id, t (s), minute, phase, type, team, player, receiver, start[x,y], end[x,y], outcome, under_pressure, ball_speed_kmh, source (sim/cv)
`StatUpdate`	match_id, t, scope (team/player), metric, value, window, delta
`Moment`	id, match_id, t, kind, importance (0–1), team, players, evidence (event ids), metrics
`Insight`	id, moment_id, segment, language, headline, body, evidence, metrics, verified, fallback, model, latency_ms
`Overlay`	overlay_id, match_id, show_at, duration, slot, priority, segment, title, body, evidence, metric
`FanProfile`	id, persona, language, club, player, metric_focus

Pitch coordinates: 105 × 68 m, origin bottom-left, home attacks left-to-right in

the 1st half. Event types: `pass, carry, shot, tackle, interception, pressure, foul, card, sub, possession_change, goal, phase_change`. Segment key format: `persona|lang|club:ID|player:ID`.

7 Repository Structure

```
pitchmind/
  contracts/            Pydantic models -> JSON Schema -> TS types
  services/
    simulator/  cv-eventing/  stats-engine/  moment-detector/
    agents/     render/       api/
  libs/
    bus/                transport adapter (local | Event Hubs)
    common/             logging, config, telemetry
  apps/
    web/                second-screen + studio dashboard (Next.js)
    overlay/            broadcast overlay page
  data/                 seed league, players, generated datasets
  infra/                Bicep modules + main.bicep
  docs/                 PLAN.tex, architecture, ADRs, demo script
  .github/workflows/    ci.yml, deploy.yml
  docker-compose.yml    full local stack
```

Conventions (empty repo, so we set them): Python `snake_case` modules, `ruff` + `mypy`, tests in `services/<name>/tests/` with pytest; TypeScript `PascalCase` components, ESLint + Prettier; structured JSON logging with `match_id`, `t` and `trace_id` on every line; errors never crash a stream consumer: log, dead-letter, continue; Conventional Commits.

8 Workstreams

Team size is flexible: assign one owner per workstream; one person may own several.

ID	Workstream	Owns
WS1	Platform & Contracts	Repo, contracts, bus adapter, docker-compose, integration
WS2	Data & Analytics	Simulator, stats engine, moment detector, xG, dataset
WS3	Agents	Agent Framework workflow, prompts, tools, Verifier, Foundry evals
WS4	Frontend	Second-screen app, studio dashboard, overlay page, RTL
WS5	Cloud & DevOps	Bicep, Azure provisioning, CI/CD, observability, Functions
WS6	CV, Docs & Video	CV module, README, architecture diagram, video, submission

9 Implementation Phases

Milestones

ID	When (Tunisia)	Gate
M1	Sat 25 Oct, 14:00	Contracts frozen; repo skeleton runs; Foundry model deployed
M2	Sun 26 Oct, 02:00	Full pipeline runs locally end to end on the demo match
M3	Sun 26 Oct, 18:00	Full pipeline runs on Azure; web app live on a public URL
M4	Sun 26 Oct, 23:00	**Feature freeze**; only bug fixes after this
M5	Mon 27 Oct, 18:00	Video uploaded; README final; repo public
M6	Mon 27 Oct, 23:00	Submitted (official cutoff Tue 28 Oct, 07:59 Tunisia)

9.1 Phase 0 – Foundations (Sat 25 Oct, 11:30–14:00) M1

1. **[WS5] Provision first, it is the slowest:** resource group, Foundry project, model deployments (one fast small model for persona/language, one stronger model for explain/recap). *Check quota now.* Event Hubs namespace + 4 hubs, Cosmos DB serverless, Web PubSub, ACR, Container Apps environment, Key Vault, App Insights.
2. **[WS1]** Create monorepo skeleton, tooling configs, `docker-compose.yml`.
3. **[WS1 + all]** Write and freeze `contracts/` (Section 6); generate JSON Schema and TS types.
4. **[WS1]** `libs/bus`: `publish(topic, model)`, `subscribe(topic, handler)`; local in-memory/Redis backend + Event Hubs backend.
5. **[WS2]** Seed data: fictional league (e.g. *North City FC* vs *Harbour United*), 22 players + bench with attributes (pace, passing, finishing, stamina, defending).
6. **[WS6]** Record own CV footage (phone video of a kickabout; no broadcast footage).

Validate: contract tests pass; `docker compose up` starts all empty services; a test message round-trips through the bus.

9.2 Phase 1 – Core engines in parallel (Sat 25 Oct, 14:00 – Sun 02:00) M2

WS2 – Simulator

- Possession state machine on a 105 × 68 pitch; per-player positions updated each event.
- Event rates and success probabilities from player attributes, pressure and fatigue.
- Realism: fatigue after 60', score-dependent tactics (trailing team pushes higher), home advantage, cards, subs.
- Seeded and reproducible; a **scripted demo seed** with a dramatic arc (early goal, pressure spell, comeback).
- Modes: live stream at a speed factor, or export a JSONL dataset.

Validate: same seed → identical output; a 90' match yields ≈ 900–1 400 events, plausible possession and pass accuracy (75–90%).

WS2 – Stats Engine

- Event-sourced state per match. Metrics: pass distance, accuracy, **difficulty rating** (distance, pressure, angle, progression), ball and shot speed, player speed and distance, xG (logistic model), pressure index (rolling opponent-half pressure), momentum (EWMA of threat), control vs chaos (possession duration, turnover rate, event entropy), rhythm (events per minute).
- Observer-based threshold watchers (sprint, shot speed, distance milestones).

Validate: unit tests with hand-computed inputs for every calculator.

WS2 – Moment Detector

- Chain of rules: goal, big chance (xG > 0.3), momentum swing, pressure surge, control ↔ chaos shift, rhythm change, sprint/speed threshold, milestone, card, substitution impact.
- Each `Moment` carries importance and evidence (event IDs + metric deltas); de-duplication window.

Validate: the demo seed produces the scripted turning points at the expected minutes.

WS3 – Agents (against recorded fixtures first)

- Agent Framework workflow: Orchestrator → Explain / Narrative / Recap → Persona → Language → Verifier.
- Tools over shared match state: `get_stats_window`, `get_events`, `get_player_profile`, `search_knowledge`.
- Structured JSON output validated against `Insight`; timeouts, one retry, template fallback.
- Verifier: rule check of every number against stats + LLM judge for unsupported claims.

Validate: 20 fixture moments → valid `Insights`; the Verifier rejects a planted wrong number.

WS4 – Frontend shell (against mock data from contracts)

- Second-screen: match header, live timeline, insight feed, persona / language / club / player selectors, stats panel.
- Overlay page: transparent background, lower-third, player tag, stat bubble slots.

Validate: pages render a mock stream; Arabic renders right-to-left.

WS1 – Integration Wire simulator → stats → moments → agents → render → web locally. *Validate (M2):* the demo match plays end to end locally with overlays appearing in the browser.

9.3 Phase 2 – Cloud and experience (Sun 26 Oct, 08:00–18:00) M3

1. **[WS5]** Bicep for all resources; Container Apps per service with KEDA scaling on Event Hubs; Managed Identity to Event Hubs / Cosmos / Foundry; secrets in Key Vault.
2. **[WS5]** Azure Function: on `phase_change` (HT/FT) trigger recap generation and archive to Blob.
3. **[WS5]** GitHub Actions: `ci.yml` (lint, type-check, test), `deploy.yml` (build, push to ACR, deploy).
4. **[WS4]** Connect to Web PubSub; segment subscription; **"Why?" button** showing evidence and metrics; player-focus mode; one-metric mode; analyst charts (momentum, xG timeline, pass map).
5. **[WS4]** Studio dashboard: grid of all active segments, hold/approve toggle (human-in-the-loop), audit trail.
6. **[WS3]** Recap agent (HT/FT, Template Method); milestones via AI Search; Content Safety on outputs.
7. **[WS3]** Foundry tracing on; Foundry evaluations: groundedness, fluency, custom numeric-fidelity eval.
8. **[WS6]** CV module: YOLO + ByteTrack on own clip, 4-point homography, ball speed, emit `MatchEvent` with `source=cv`.

Validate (M3): a public URL shows a live demo match; App Insights shows traces across all services; latency budget met.

9.4 Phase 3 – Hardening (Sun 26 Oct, 18:00–23:00) M4

- **Failure-recovery demo:** a switch that kills the Narrative Agent mid-match; show fallback text and recovery.
- Throttling and priority so overlays never flood the screen (max one lower-third per 20 s).
- Load test: 200 active segments; measure end-to-end latency and cost per match.
- **STRETCH**: expose the Verifier as an A2A endpoint; Azure AI Speech voice commentary; publish the dataset in the repo.
- Feature freeze at 23:00.

9.5 Phase 4 – Submission (Mon 27 Oct) M5, M6

1. README: problem, architecture diagram, Microsoft technologies list, how to run locally and on Azure, eval results, cost model.
2. Record and edit the video (Section 10). Royalty-free music or none.
3. Scan for secrets, then make the repository public.
4. Write the pitch; fill the project page; link video and repo; **submit early**.

10 Demo Video Script (under 2:00)

Time	Shot
0:00–0:12	Problem: stats show *what*, not *why*; every fan is different
0:12–0:30	Live synthetic match starts; events stream; stats update (**Ingest, Interpret**)
0:30–0:50	Pressure surge detected; overlay explains why, "Why?" reveals evidence (**Explain, Render**)
0:50–1:10	Same moment, three fans side by side: analyst EN, casual AR, beginner FR (**Personalize**)
1:10–1:22	Player-focus mode and speed tag from the CV clip
1:22–1:35	Kill an agent live; fallback keeps the overlay alive; Verifier blocks a wrong number
1:35–1:50	Architecture: Event Hubs, Container Apps, Functions, Web PubSub, Foundry, Agent Framework
1:50–1:58	Half-time recap auto-generated; closing line

11 Testing and Validation

Level	What
Contract tests	Every service's input/output validates against `contracts/`
Unit tests	Every stats calculator and detection rule with hand-computed cases
Determinism	Same simulator seed → byte-identical event stream
Agent tests	Mocked LLM for routing/fallback logic; fixture moments for real-model smoke tests
Evaluations	Foundry evals on 50 generated insights: groundedness, fluency, numeric fidelity
E2E	Playwright: open app, pick persona/language, receive an overlay within budget
Cloud smoke	Post-deploy script runs a 5-minute match at 20× and checks App Insights

```
ruff check . && mypy services libs contracts
pytest -q
npm --prefix apps/web run lint && npm --prefix apps/web run test
npx playwright test
```

12 Risks

Risk	Level	Mitigation
Time: 2 days for full scope	**HIGH**	Strict milestones; feature freeze 23:00 Sunday; video day protected
Azure model quota / region availability	**HIGH**	Provision in Phase 0; have a second region ready
Integration breaks late	**HIGH**	Contracts frozen at M1; integrate continuously, not at the end
LLM latency breaks the "live" feel	**MED**	Small model for persona/language; parallel fan-out; segment cache; template fallback
Hallucinated stats on screen	**MED**	Evidence-only prompts, Verifier rule check, reject on mismatch
Azure cost / credit exhaustion	**MED**	Serverless Cosmos, scale-to-zero Container Apps, budget alert
CV accuracy weak	**MED**	Scope to one clip and ball speed; presented as a plug-in source
Trademark / copyright in video	**MED**	Fictional clubs, own footage, no music or royalty-free only
Secrets leaked when repo goes public	**MED**	Key Vault + Managed Identity; secret scan before making public
Arabic RTL layout bugs	LOW	Test early with mock data

13 Complexity Estimate

Part	Complexity	Effort (person-hours)
Contracts, repo, bus	Low	4
Simulator + dataset	Medium	8
Stats engine + xG	Medium	8
Moment detector	Medium	5
Agents + Verifier + evals	High	14
Render service	Low	3
API / BFF	Low	4
Second-screen + studio + overlay	High	14
Azure, Bicep, CI/CD, observability	High	10
CV module	Medium	6
README, video, submission	Medium	6
Total	**High**	**≈ 82 person-hours**

14 Definition of Done

- ☐ All 5 pipeline stages run on Azure on a public URL.
- ☐ Demo match plays with overlays, "Why?" evidence, 3 personas × 4 languages, player-focus mode.
- ☐ HT and FT recaps generated automatically.
- ☐ Failure recovery and Verifier rejection shown live.
- ☐ CV clip produces at least one event in the same pipeline.
- ☐ Tests and CI green; Foundry evaluation results in the README.
- ☐ Repo public, no secrets; video under 2:00 and public; pitch written.
- ☐ Submitted by Mon 27 Oct, 23:00 Tunisia time (cutoff Tue 28 Oct, 07:59).